Il primo programma per il PIC: MPLAB X, TRIS e PORT

Lezione 4 — 3MAK, SST-INF-01c — Architettura, configurazione, numeri esadecimali e il primo lampeggio sul simulatore

22/10/2026

Nota

Finora (dispense A01–A03) abbiamo visto **variabili**, **tipi** e **operatori**, e i costrutti **if**, **for** e **while**. Oggi ci agganciamo alla **programmazione del microcontrollore**: non più programmazione generica, ma programmi che comandano davvero i **piedini** del **PIC16F628A**. Usiamo **MPLAB X** per scrivere e compilare e **PICSimLab** per simulare. In fondo trovate come **installare gli strumenti** a casa e il **compito per casa** (C01).

1 Com'è fatto un microcontrollore

1.1 CPU e due memorie: l'architettura Harvard

Un **microcontrollore** si può schematizzare come un rettangolo che contiene una **CPU** (*Central Processing Unit*, l'unità che **fa i calcoli**) e **due memorie**:

- la **program memory**, dove mettiamo il nostro **programma**;
- la **data memory**, dove mettiamo i **dati** da elaborare.

Questa organizzazione si chiama **architettura Harvard**. Se vale la pena darle un nome, vuol dire che ne esiste almeno un'altra: l'**architettura di Von Neumann**, in cui c'è la CPU e **una sola memoria** che contiene sia il programma **sia i dati**.

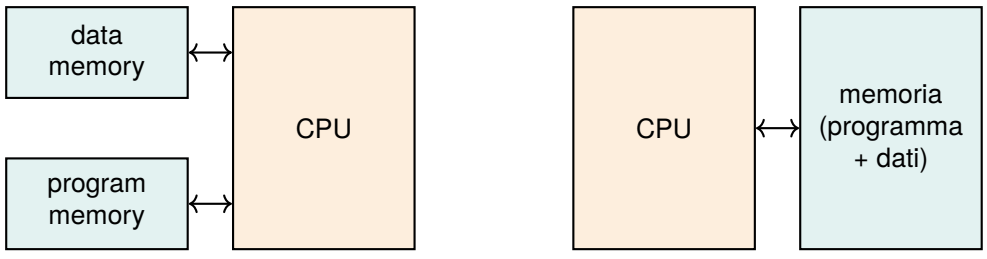


Figura 1: Le due architetture disegnate alla lavagna: a sinistra due memorie separate, a destra una sola memoria per programma e dati.

- Il **PC** che avete davanti usa la **Von Neumann**: c'è la CPU (ad esempio un Intel i7) e la **RAM**, vista come **un'unica memoria** in cui stanno sia i programmi sia i dati.

- Il **PIC** di Microchip (ma anche l'Arduino, cioè il suo microcontrollore) usa l'**Harvard**: una memoria in cui si **scarica il programma** e un'altra per i **dati** su cui lavorare.

Qual è meglio? Come prestazioni l'**Harvard**, perché permette di **accedere contemporaneamente** al programma e ai dati: mentre legge l'istruzione può già lavorare sui dati. Con la Von Neumann c'è **una sola strada** per programma e dati, quindi a parità di tecnologia le operazioni risultano più lente. Allora perché i computer usano la Von Neumann? Per una questione di **costi**: la memoria veloce dell'Harvard costa di più **per ogni singolo byte**, e un PC ha bisogno di **miliardi** di byte. (Poi i PC recuperano velocità in molti altri modi: un PC moderno è comunque enormemente più veloce di un microcontrollore.)

1.2 Quanta memoria? PC, Arduino e PIC a confronto

Dispositivo	RAM (dati)	Costo indicativo
PC con 16 GB	16 miliardi di byte (16 miliardi di "cassetti")	processore i7: 400–500 euro
Arduino Uno (ATmega328)	2 KB, cioè circa 2000 byte	
PIC16F628A	**224 byte**	circa 3 euro

I 16 GB di un PC sono circa **8 milioni di volte** i 2 KB di un Arduino, e il PIC ne ha ancora meno: in 224 byte non ci starebbero nemmeno i nomi di tutti gli allievi della classe. Però per far funzionare una **lavatrice** o un qualunque dispositivo che non sia un computer, sono **più che sufficienti**. Oggi la RAM per PC costa parecchio perché la **domanda** è molto alta: se tanti clienti chiedono tanta RAM e le aziende non riescono a starci dietro, i **prezzi salgono**.

1.3 Le periferiche: i GPIO

Oltre alle memorie, il microcontrollore ha delle **periferiche**, che si occupano di **buttare fuori** i dati o di **acquisirli** attraverso i **piedini**. Quelle che oggi ci interessano si chiamano **GPIO** (*General Purpose Input/Output*, ingressi/uscite di uso generale): è una parola che troverete **tantissimo** nei manuali dei microcontrollori. Servono per **accendere un LED**, piuttosto che **rilevare un'informazione** dall'esterno (un sensore che rileva la presenza di qualcuno, un pulsante premuto…).

2 TRIS e PORT: comandare i piedini

Ogni microcontrollore ha almeno **due piedini di alimentazione** (sul PIC16F628A: **+5 V** e **GND**, come nello schema elettrico disegnato la volta scorsa). Gli altri piedini si possono **accendere o spegnere** come meglio crediamo, e a livello logico sono raggruppati **a gruppi di 8**: il **gruppo A** (piedini RA0…RA7) e il **gruppo B** (RB0…RB7).

A ciascun gruppo fanno capo dei **registri**. Un **registro** è praticamente una **variabile** che possiamo modificare: una casellina da **8 bit**, numerati da **0 a 7**, uno per ogni piedino del gruppo.

Definizione

- **TRIS** (**TRISA**, **TRISB**) **definisce la direzione** dell'informazione: per ogni bit, **1** = il piedino è un **ingresso** (input), **0** = è un'**uscita** (output). Trucco per ricordarlo: l'**1** somiglia alla **I** di Input, lo **0** alla **O** di Output.
- **PORT** (**PORTA**, **PORTB**) **rappresenta l'informazione**: se il piedino è un'uscita, scrivendo **1** lo accendo (se c'è un LED, il LED si accende), scrivendo **0** lo spengo; se è un ingresso, leggendo PORT scopro che cosa arriva da fuori.

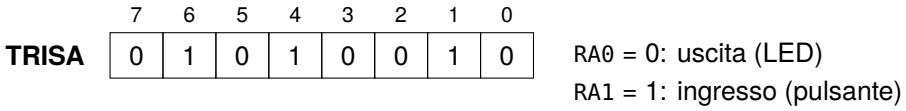


Figura 2: L'esempio di TRISA alla lavagna (`0b01010010`): il bit 0 vale 0, quindi RA0 è un'uscita a cui posso collegare un LED; il bit 1 vale 1, quindi RA1 è un ingresso a cui posso collegare un pulsante. In tutto sono uscite RA0, RA2, RA3, RA5 e RA7; ingressi RA1, RA4 e RA6.

Esempio

Se voglio **RB4** come **uscita**, in **TRISB** il **bit 4** deve valere **0**. Se voglio **RB6** come **ingresso** (per collegarci un pulsante), il **bit 6** di TRISB deve valere **1**. Con un LED su un'uscita sono io, microcontrollore, a **decidere** che cosa deve fare; con un pulsante su un ingresso è qualcuno da fuori che lo preme, e io devo **stare ad ascoltare** se e quando viene premuto.

2.1 Scrivere e leggere PORT

- **Scrittura**: basta mettere PORT **a sinistra dell'uguale**. Ad esempio `PORTA = 0b11111111;`. La scrittura ha effetto solo sui bit **che sono uscite**: con il TRISA dell'esempio sopra, andranno alti solo i piedini che hanno 0 in TRISA, cioè i bit **0, 2, 3, 5 e 7** (RA0, RA2, RA3, RA5, RA7), mentre gli ingressi **se ne fregano** di quello che ho scritto, perché un ingresso non è sensibile a una scrittura. (In realtà non è proprio così, ma per ora ragioniamo in questi termini; poi miglioreremo.)
- **Lettura**: PORT in **qualunque posizione** che non sia a sinistra di un uguale. Ad esempio `z = PORTA;` legge la porta e salva il valore nella variabile `z`, per elaborarlo dopo.

Esempio

L'analogia della porta d'ingresso. Immaginate un locale con due porte: da una **si entra solo**, c'è un tornello, dall'altra **si esce solo**. Con **TRIS** decido quale porta è l'ingresso **e quale l'uscita**. Se devo **accendere un LED** devo comunicare **dall'interno verso l'esterno**, quindi mi serve una porta d'uscita: se provo a uscire da una porta d'ingresso (scrivere 1 su un piedino che è un ingresso), **non riesco a uscire**, e il LED non si accende. Su un ingresso invece **leggo**: "fammi vedere chi sei, per poterti valutare".

3 Gli strumenti: MPLAB X e PICSimLab

Usiamo due programmi:

- **MPLAB X**: l'**IDE**, cioè il programma con cui **scriviamo e compiliamo** i programmi. È il software proprietario di **Microchip**, il produttore dei PIC (il codice si potrebbe scrivere in qualunque editor di testo, ma MPLAB X fa tutto il resto);
- **PICSimLab**: un **simulatore** della scheda con il PIC, che per ora usiamo al posto del circuito vero.

3.1 Creare un nuovo progetto

1. All'avvio, chiudete le pagine che non servono (*Start Page*, *MPLAB X Store*): deve restare la scheda **Projects**.
2. Pulsante **arancione** in alto a sinistra, **New Project**. Scegliete **Microchip Embedded** → **Application Project(s)** e **Next** (le altre voci non ci interessano).
3. **Device**: è il microcontrollore che usiamo, il **PIC16F628A**. **Attenzione alla A finale**: l'elenco mostra sia il 628 sia il **628A**, prendete quello con la A. Se cambiate microcontrollore dovete cambiare il device, coerentemente. (Togliendo il filtro di ricerca si vede che i device sono tantissimi: noi per ora ci limitiamo a questo, piccolo e semplice.)
4. **Tool**: nessun debugger, **Next**.
5. **Compilatore**: è il programma che prende il codice che scriviamo noi (`PORTA = ...`, `if`, `else`, `while`, `for...`) e lo **converte in numeri binari**, codici numerici che la macchina sa interpretare. Questi numeri fanno funzionare le **porte logiche** interne (come le porte AND viste in prima: premo due pulsanti e si accende), che eseguono il programma. Le opzioni sono tre:
   - **XC8**: **è quello che usiamo**;
   - **HI-TECH**: non è installato, quindi non è selezionabile;
   - **pic-as**: l'**assembler**, cioè si scrive il codice esattamente nel **linguaggio nativo** della macchina. Il docente ha imparato così; oggi ci sono tecniche molto più strutturate e semplici.

   In aula il PC del docente ha XC8 versione 3.10, quelli del laboratorio la 3.00: non cambia niente.
6. **Nome e cartella del progetto**: scegliete dove salvarlo e dategli un nome, ad esempio `02-SecondoEsempio`. Usate solo **lettere e numeri**: **niente spazi**, **niente punti** e niente simboli particolari (anche il trattino è meglio evitarlo). Un nome con caratteri strani **manda in palla il compilatore**: la macchina poi non compila e vi dice solo che non funziona. Più corto è il **nome, meglio è**. Poi **Finish**.

Nota

Nella parte sinistra, alla voce **Memory**, MPLAB X mostra le due memorie dell'architettura Harvard: per il PIC16F628A **Data 224 byte** e **Program 2048** parole, cioè non possiamo fare programmi più lunghi di circa 2048 istruzioni. Per far lampeggiare un LED sono ovviamente sufficienti; un circuito che fa tante cose (comunica, gestisce un display...) potrebbe non starci. Il docente ha imparato sul **PIC16F84**, negli anni '90: 68 byte di RAM, 1024 istruzioni di programma, tutto in assembly. A un certo punto la memoria finì e servirono "mille accrocchi" per ridurre il programma; cambiare microcontrollore voleva dire buttare tutto e ripartire da capo, perché ogni microcontrollore ha il suo manuale da studiare. Quando compilate, due **barre gialle** mostrano quanto avete riempito delle due memorie.

3.2 Il file `main.c`

Nella parte superiore c'è un'organizzazione in **cartelle virtuali** del progetto (*Header Files*, *Source Files*...): se andate a cercare il progetto sul disco **non le trovate**, sono solo un modo con cui MPLAB X gestisce i file. Click destro sulla cartella virtuale **Source Files** → **New** → **main.c** (proprio `main.c`, non `avr-main` o altro). Si apre una finestra che chiede il nome del file: il docente lo chiama sempre `main`; potete chiamarlo come volete, "anche Gertrude", ma senza punti, simboli **strani, slash, asterischi**. **Finish**.

Attenzione

Se il progetto non compila per un nome sbagliato, ad esempio con uno **spazio**: si apre il file, si copia tutto il codice, si **rimuove** il file dal progetto, si crea un nuovo `main.c` con un nome corretto e si incolla di nuovo il codice.

3.3 Il blocco `main`

In C c'è una regola assoluta e inderogabile: ogni programma deve avere il blocco funzionale **`main`**. Le due **graffe** di `main` definiscono un **contenitore**, una scatola delimitata: il nostro codice deve stare **dentro** la scatola, non sopra né sotto. Le uniche eccezioni sono alcune righe speciali che vanno **in cima al file**, prima di `main`: le vediamo subito.

4 Le direttive di configurazione

Quando la macchina viene accesa ha bisogno di sapere **come funzionare**, come noi appena svegli: prima di fare qualunque cosa deve partire il cuore, si deve cominciare a respirare. La macchina deve sapere qual è il suo **"cuore"**, il motore che manda avanti le operazioni (che tipo di **clock** usa: un oscillatore RC, uno interno, un **quarzo**...), e se deve attivare certi **sistemi di sicurezza**. Queste **direttive** si preparano con uno strumento di MPLAB X:

1. menu **Window** → **Target Memory Views** → **Configuration Bits**;
2. nella finestra che si apre in basso si scelgono le **opzioni** nella colonna *Option*;
3. si preme **Generate Source Code to Output**: MPLAB X produce un testo, di cui ci interessano solo le **8 righe** che iniziano con `#pragma`;

4. si copiano (Ctrl+C) e si incollano **in cima** al file `main.c`.

Definizione

Le opzioni da impostare (regola generale: **HS**, **off**, poi **tre ON**, poi **tre OFF**):

- **FOSC = HS**: l'oscillatore di sistema. HS sta per *High Speed*, quarzo **sopra i 4 MHz**. Storicamente le macchine andavano fino a 4 MHz (non più di 4 milioni di operazioni al secondo); quando sono arrivate quelle da 5, 6, 8 MHz e oltre, si è distinto tra bassa velocità e alta velocità. Oggi 4 MHz sono veramente pochi: un PC con un i7 di prima generazione (di circa 16 anni fa) lavora a circa **2,3 GHz**, cioè 2 miliardi e 300 milioni di cicli al secondo, quasi **600 volte** i 4 MHz. I PIC che usiamo noi arrivano al massimo a **20 MHz**. Quindi per noi **sempre HS**.
- **WDTE = OFF**: il **watchdog**, il "cane da guardia". È un sistema di sicurezza che **resetta la macchina** se si blocca. Immaginate il computer di un aereo che, una volta ogni milione, si impalla per un errore di programmazione: non possiamo permettercelo, quindi è meglio che si **resetti** e riparta da capo, magari perdendo il controllo per pochi secondi. È complicato da usare: non sapendolo ancora impostare, **lo disabilitiamo**.
- **PWRTE = ON**: il *Power-up Timer* va **abilitato**. Il perché lo vedremo più avanti.
- **MCLRE = ON** e **BOREN = ON**: restano accesi.
- **LVP = OFF**: la *Low-Voltage Programming* va **spenta**.
- **CPD = OFF** e **CP = OFF**: sono le **protezioni** della memoria dati e del codice. Se inventate un dispositivo che solo voi sapete fare, il programma è il vostro vantaggio: qualcuno potrebbe prendere il microcontrollore, collegare due sonde, **estrarre il codice**, rifare il circuito e venderlo a un decimo del prezzo. Con **CP** (*Code Protection*) attivo il codice **non si può più leggere** da fuori, e con **CPD** lo stesso vale per i dati. A noi non interessa: **li teniamo spenti**.

```
#pragma config FOSC = HS
#pragma config WDTE = OFF
#pragma config PWRTE = ON
#pragma config MCLRE = ON
#pragma config BOREN = ON
#pragma config LVP = OFF
#pragma config CPD = OFF
#pragma config CP = OFF

#include <xc.h>

void main(void)
{
    // configureremo il gruppo A e B

    // decidiamo chi accendere e chi spegnere

    return;
}
```

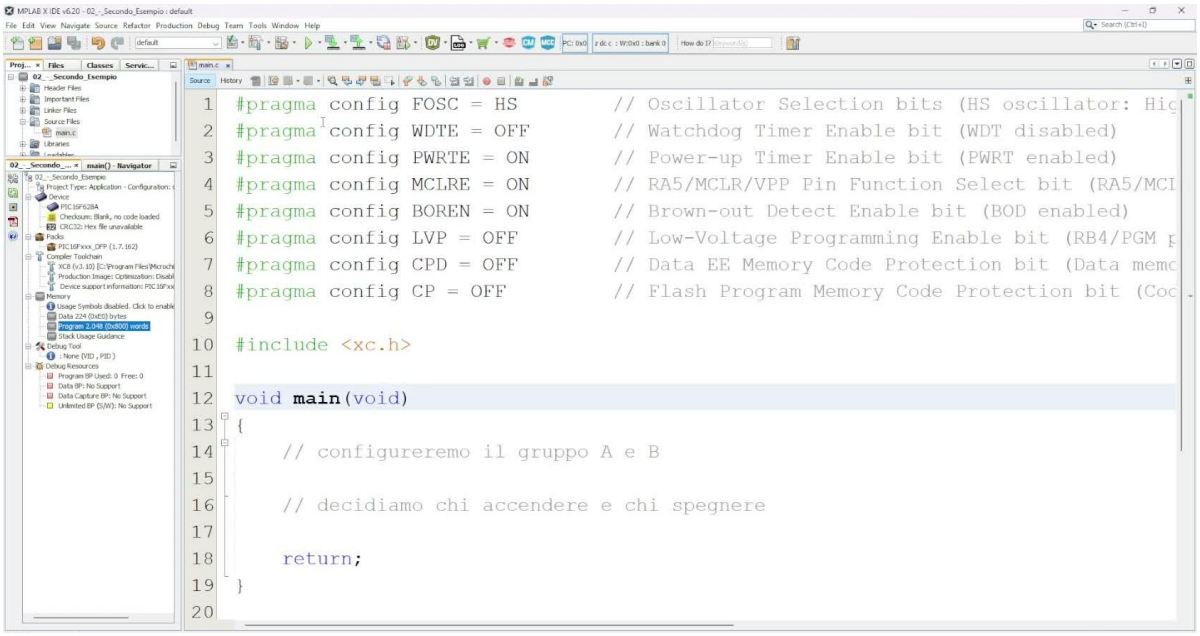


Figura 3: Il progetto in MPLAB X v6.20: in cima le 8 righe `#pragma config` generate dalla finestra Configuration Bits, poi `#include <xc.h>` e il blocco `main`. A sinistra, sotto *Memory*: Data 224 byte, Program 2048 parole.

Nota

Le 8 righe `#pragma` sono **sempre uguali**: se ve le salvate da qualche parte, pronte da copiare, non dovete rifare ogni volta tutta la procedura. Se non le avete salvate, dovete rifarla ricordandovi cosa va acceso e cosa spento. Il docente queste impostazioni ha dovuto scoprirle da solo, provando tutte le combinazioni possibili: “ci ho messo due mesi”.

4.1 Compilare: il pulsante Build

Compilare vuol dire convertire il programma in un codice che il microcontrollore può eseguire. Fino alla versione **6.15** di MPLAB X bastava il pulsante con il **martello** in alto (il martello con lo scopino, invece, **cancella tutto e ricompila** da capo). Dalla **6.20** quel pulsante fa una cosa simile ma che a noi non va bene: bisogna cliccare sulla **freccina** accanto e scegliere la seconda voce, **Build Main Project**. Se è tutto a posto, in basso compare in **verde** `BUILD SUCCESSFUL`.

Esempio

Un pulsante comodo per Build. Click destro sulla zona vuota della barra in alto → **Customize** (ultima voce). Nella finestra: **New Toolbar**, date un nome qualunque (il docente la chiama "Mauro") e OK: la nuova barra compare in alto a destra. Poi cercate nell'elenco, un po' dopo la metà, l'icona **Build Project** e **trascinatela** nella nuova barra. Da quel momento un click compila direttamente.

Compilando il programma vuoto, a sinistra le barre di memoria mostrano **1%** di memoria dati (2 byte, usati dalla configurazione) e **0** di programma, perché non abbiamo ancora scritto niente.

5 Un programma che non finisce mai

Quando la macchina si accende, legge le direttive per capire come funziona e poi la **prima cosa** che fa è eseguire **`main`**. E quando ha finito `main`, che cosa fa? **Si ferma**. Su un PC un programma che finisce va benissimo, ma in un **dispositivo embedded** no:

- un **pacemaker** che arriva alla fine del programma si ferma e resta bloccato finché non lo si spegne e riaccende;
- un **frigorifero** con la scheda programmata per "raffreddare finché arrivi a 4 gradi" accende il compressore, arriva a 4 gradi e finisce il programma: il compressore non ripartirà più e la temperatura risalirà; l'unico modo per farlo funzionare di nuovo è staccare e riattaccare la spina;
- anche nel cellulare, WhatsApp è sempre lì a disposizione: non finisce dopo un messaggio.

Definizione

Nei dispositivi elettronici il **programma non deve mai finire**. Per questo il codice che deve funzionare sempre si mette dentro un **ciclo infinito**: un `while` con una **condizione sempre vera**.

Abbiamo detto (dispensa A03) che si preferisce il **for** quando si sa già quante volte ripetere, il **while** quando non si sa. Qui non dobbiamo finire mai, quindi usiamo il `while` con una condizione **che non sarà mai falsa**:

- `while (1 + 1 == 2)` funziona, ma sono tanti caratteri; `while (1 > 0)` ancora meglio;
- ricordate che l'unico numero **falso** è lo **zero**: qualunque altro numero è vero. Quindi va bene `while (5)`, `while (42)`, `while (-426)`... ma **non** `while (0)`. La scrittura tipica, quella che userà sempre il docente, è **`while (1)`**;

- non mettete mai una condizione **variabile** che ogni tanto potrebbe diventare falsa: il programma finirebbe e la macchina si bloccherebbe;
- chi "è anticonformista" può usare `for (;;)`: un for senza variabile di controllo, senza condizione di uscita e senza incremento ripete per sempre, e ha un carattere in meno di `while(1)`.

Nota

E Arduino? L'anno scorso il programma aveva due funzioni, `setup` e `loop`, e nessun `main`. In realtà il `main` c'è, scritto nei file di Arduino: chiama `setup` una volta e poi chiama `loop` **dentro un ciclo infinito**. `loop` non si ripete "perché si chiama loop", ma perché è dentro un `while` infinito. Noi invece scriviamo tutto in modo diretto: **prima del `while`** quello che si fa una volta (impostare chi è input e chi è output), **dentro il `while`** quello che si ripete (accendere, spegnere, leggere un dato, attivare una sirena...).

6 Il primo programma: accendere un LED

6.1 La scheda simulata

In **PICSimLab** scegliete dal menu **Board** la scheda **McLab1** (l'altra finestra che si apre si può chiudere). La scheda ha **8 LED** collegati alla **porta B** (RB0...RB7) e **4 pulsanti** sulla porta A (RA1...RA4). Poiché sulla porta B ci sono solo LED, tutta la porta B va **impostata come uscita**: è vero che posso scegliere quello che voglio, ma impostare come ingresso un piedino collegato a un LED non avrebbe senso. Il simulatore è una scatola chiusa in cui non possiamo cambiare niente; su un circuito vostro farete le scelte che servono.

Per avere tutti i bit di TRISB a **0** (output) devo scrivere il numero che ha **8 bit a 0**, cioè **0**:

```
// questa assegnazione imposta tutti i pin B come uscite
TRISB = 0;
```

6.2 Ripasso: da binario a decimale

Per convertire un numero binario in decimale si dà un **peso** a ogni cifra, partendo da destra: $2^0, 2^1, \ldots, 2^7$, e si sommano **solo** i pesi delle cifre che valgono 1. Esempio fatto in aula con la sequenza `11000001`: $128 + 64 + 1 = \mathbf{193}$.

bit	7	6	5	4	3	2	1	0
peso	128	64	32	16	8	4	2	1
`11000001`	1	1	0	0	0	0	0	1

6.3 Accendere RB4

Vogliamo accendere il LED su **RB4**: in PORTB il **bit 4** deve valere 1 e tutti gli altri 0. Il bit 4 pesa $2^4 = \mathbf{16}$, quindi:

```c
#include <xc.h>

void main(void)
{
    // questa assegnazione imposta tutti i pin B come uscite
    TRISB = 0;

    while (1)
    {
        // accendo o spengo o leggo un dato
        PORTB = 16;
    }
}
```

(Le righe `#pragma config` in cima restano sempre: qui e negli esempi successivi non le ripetiamo.)

Attenzione

Errori visti alla prima compilazione:

- è rimasta nel codice la parola `loop`, scritta solo per spiegare il discorso di Arduino: va cancellata;
- `PORTB = o;` con la **lettera o** al posto dello **zero**.

"Sugli sbagli si costruisce la sapienza."

6.4 Caricare il programma nel simulatore

1. Compilate. Nell'output, sotto `BUILD SUCCESSFUL`, c'è una riga `Loading code from` seguita dal **percorso** del file prodotto. Selezionate il percorso **fermandovi alla prima cartella `production`**: quello che viene dopo è il nome del file, che non si può mettere nella barra di navigazione. Ctrl+C.
2. In PICSimLab: **File** → **Load Hex**. Si apre una finestra.
3. Cliccate nella **barra dell'indirizzo** in alto, sulla parte non scritta, in modo da selezionare tutto, e fate **Ctrl+V**. Poi premete **Invio**.
4. Ora vedete un unico file con il nome del vostro progetto e l'estensione **`.hex`**: è il programma compilato. Apritelo.

Attenzione

L'errore tipico è fare Ctrl+V e poi premere **Apri**: così non funziona. Bisogna incollare il percorso nella barra, premere **Invio** e solo dopo scegliere il file `.hex`.

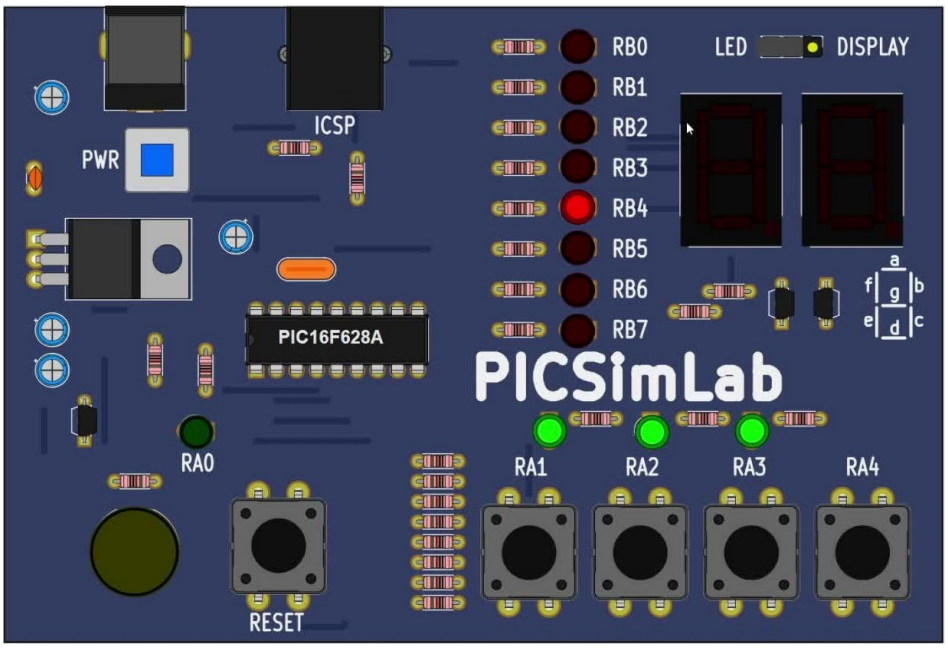


Figura 4: PICSimLab, scheda McLab1 con il PIC16F628A: il LED RB4 è acceso. In alto a destra il deviatore LED/DISPLAY, a sinistra il campo *Clk (MHz)*.

Attenzione

"A nessuno funziona!" La scheda ha un **deviatore LED/DISPLAY**: se è su **DISPLAY**, i piedini della porta B vanno ai due **display a 7 segmenti** invece che ai LED. RB4 è proprio il piedino che **non è collegato ai segmenti**, quindi accendendo RB4 con il deviatore su DISPLAY non si vede niente. Spostando il deviatore su **LED**, il LED RB4 si accende.

6.5 Accendere tre LED

Esercizio prima della pausa: accendere **RB1**, **RB4** e **RB6**. Bisogna mettere un 1 nei bit 1, 4 e 6: $2 + 16 + 64 = \mathbf{82}$, quindi `PORTB = 82;`.

7 Scrivere i numeri in binario e in esadecimale

Ogni volta convertire a mano, scegliere le caselle e sommare i pesi è **oggettivamente scomodo**. Ci sono due strade più comode.

7.1 Il prefisso `0b`: il numero in binario

La prima strada, la più semplice ma quella che produce **più errori**, è scrivere il numero **direttamente in binario**. Basta mettere davanti un **prefisso**, come il **prefisso del telefono**: +39 è il prefisso internazionale dell'Italia, 0434 è Pordenone, 0432 Udine, 0422 Treviso. Una volta anche nei cellulari il prefisso indicava l'**operatore** (ad esempio 338 e 335 TIM, 329 Vodafone, 333 Wind) e cambiare operatore voleva dire **cambiare numero** e perdere i contatti; da quando c'è la portabilità del numero quel legame non c'è più. In C il prefisso che dice "questo numero è binario" è **`0b`**:

```
// RB1, RB4, RB6 accesi: 82 scritto in binario
PORTB = 0b01010010;
```

Alla lavagna l'esempio era `PORTB = 0b11010010;` (accesi RB7, RB6, RB4, RB1): è lo stesso numero che tra poco scriveremo in esadecimale. Non serve pensare per convertire: si scrive il numero esattamente com'è. Il problema è che con un microcontrollore a 8 bit si scrivono 8 cifre, ma su una macchina a **32 bit** si dovrebbero scrivere 32 cifre: la probabilità di **sbagliare il numero di cifre** è alta, e il numero diventa illeggibile.

7.2 Il sistema esadecimale

Ci viene in aiuto una codifica che non è la decimale e non è la binaria: l'**esadecimale**, che usa **16 simboli** diversi. In decimale usiamo 10 simboli (da 0 a 9), in binario 2 (0 e 1). Per arrivare a 16 servono **6 simboli nuovi** dopo il 9.

Nota

I simboli sono una scelta arbitraria. Alla lavagna abbiamo inventato due simboli a caso e li abbiamo chiamati "CNS" e "parapapà", e abbiamo detto che valgono 10 e 11: allora "parapapà + CNS" fa 21. Chi ha deciso che il simbolo 4 si chiama "quattro" e si disegna così? È una **decisione storica dell'umanità**: in altri paesi lo chiamano con un altro nome, e anche il 4 si può scrivere in due modi diversi. Una volta imparato a riconoscere i simboli, la matematica costruisce tutto il resto. Per comodità, nell'esadecimale si usano le lettere: **A = 10**, **B = 11**, **C = 12**, **D = 13**, **E = 14**, **F = 15**.

Come si conta. È il concetto del **pallottoliere** (l'abaco): quando ho finito le cifre, aggiungo una cifra a sinistra e riparto da 0. In decimale dopo 9 viene 10, dopo 99 viene 100. In binario le cifre finiscono subito, quindi il numero di cifre cresce in fretta: 0, 1, 10, 11, 100, 101… In base 4 (cifre 0–3): 0, 1, 2, 3, 10, 11, 12, 13, 20… 33, 100. In esadecimale: 0…9, A…F, poi 10, 11…19, 1A…1F, 20… fino a 9F, A0… FF, e dopo FF viene **100**.

dec	bin	base 4	hex	dec	bin	base 4	hex
0	0000	0	0	8	1000	20	8
1	0001	1	1	9	1001	21	9
2	0010	2	2	10	1010	22	A
3	0011	3	3	11	1011	23	B
4	0100	10	4	12	1100	30	C
5	0101	11	5	13	1101	31	D
6	0110	12	6	14	1110	32	E
7	0111	13	7	15	1111	33	F

Un numero esadecimale si legge con la stessa **notazione posizionale** del decimale, ma con pesi 1, 16, 256…: ad esempio `6B9` vale $6 \times 256 + 11 \times 16 + 9 = 1721$.

7.3 Il trucco: convertire a fette di 4 bit

Convertire da binario a decimale è scomodo: guardando 82 non si vede che rapporto c'è con i bit. Quando invece si passa tra basi che sono **potenze di 2** (2, 4, 8, 16), si può convertire il numero **a fette**, indipendentemente dal resto del valore. Per l'esadecimale le fette sono da **4 bit** (un *nibble*):

ogni gruppo di 4 bit corrisponde a **una cifra esadecimale**. Con 7 o 10 simboli (che non sono potenze di 2) questo **non si può fare**.

Esempio

`11010010` si divide in `1101` e `0010`. Dalla tabella: `1101` = **D**, `0010` = **2**. Quindi `11010010` = **`0xD2`** (il prefisso esadecimale in C è **`0x`**). In decimale vale 210.

```
// 0xD2 = 1101 0010: accesi RB7, RB6, RB4, RB1
PORTB = 0xD2;
```

Compilando e caricando nel simulatore, partendo dall'alto: RB7 acceso, RB6 acceso, RB5 spento, RB4 acceso, RB3 e RB2 spenti, RB1 acceso, RB0 spento.

Attenzione

Se scrivete un numero con le lettere **senza** `0x` (ad esempio `D2`), il compilatore dà errore, perché un numero decimale non può contenere una D. Il pericolo vero è un numero esadecimale **fatto solo di cifre**, come `25`: se dimenticate `0x`, il compilatore lo prende per il 25 decimale, **non dà nessun errore**, ma il valore non è quello che volevate.

Definizione

Che scriviamo in decimale, in binario o in esadecimale, dentro la macchina il numero è **sempre in binario**. Decimale (nessun prefisso), `0b` e `0x` sono solo modi diversi di scrivere **lo stesso numero**. L'esadecimale è la scrittura più comoda e quella che usano tutti i programmatori: la tabellina ha 16 simboli, i primi 10 li conosciamo già, bisogna imparare solo da A a F. Il docente scrive sempre in esadecimale; voi potete scegliere, ma scrivere in binario è **molto inefficiente**.

7.4 Esadecimale nella vita di tutti i giorni

- **Il codice ASCII.** Anche quando scriviamo un tema in Word, ogni lettera viene salvata in memoria come un **numero**. Il **codice ASCII** è la tabella che lega i numeri ai caratteri: ad esempio la lettera **A** è il numero **65** (in esadecimale `0x41`). Prima che ci si mettesse d'accordo, un testo scritto su un computer e aperto su un altro di marca diversa poteva risultare illeggibile, perché i caratteri corrispondevano a numeri diversi: è stata una piaga per anni.
- **Internet e le decisioni del passato.** Internet è figlio di tantissime decisioni prese nel tempo, per restare **compatibili** con le macchine vecchie: le aziende inventavano tecnologie più moderne, ma dovevano funzionare anche con quello che la gente aveva già, perché nessuno butta via tutto per comprare il nuovo. Per questo i protocolli sono pesanti: ogni scambio di dati porta con sé intestazioni e messaggi "di servizio" (aprire la connessione, salutarsi, chiuderla), e per un messaggio piccolo la parte di servizio può essere **molto più grande** dei dati utili. L'esempio fatto in aula, per dare l'idea: se per mandare un testo di 100 lettere servissero 4000 caratteri per "dire buongiorno" e altri 4000 per "dire arrivederci", avrei speso 8100 caratteri, di cui **utili solo 100**. Se si potesse riprogettare la rete da zero, a parità di caratteristiche sarebbe molto più efficiente.
- **Gli indirizzi IP.** Un indirizzo **IPv4** (lo trovate nelle impostazioni di rete di Windows) è fatto di **4 numeri da 8 bit**, quindi ognuno va da 0 a **255**: 255.255.255.255 è il più grande. In tutto sono 32 bit, circa **4 miliardi** di indirizzi, ma i dispositivi collegati sono molti di più. Il problema

è stato aggirato con le **reti private** (molti dispositivi dietro un solo indirizzo pubblico, con la **maschera di sottorete** che divide la rete) e risolto con gli indirizzi **IPv6**, da **128 bit**. Scritti in decimale o in binario sarebbero lunghissimi (128 cifre binarie!), per questo si scrivono in **esadecimale**: **8 gruppi da 16 bit**, ognuno scritto con 4 cifre esadecimali e separati da due punti. Un gruppo da 16 bit va da 0 a **65535** (pesi $2^0 = 1, \ldots, 2^{10} = 1024$, $2^{11} = 2048$, $2^{12} = 4096$, $2^{13} = 8192$, $2^{14} = 16384$, $2^{15} = 32768$), cioè da `0000` a `FFFF`: ad esempio un indirizzo che inizia con `fe80` (`fe80 = 1111 1110 1000 0000`). Quando si collega un nuovo dispositivo (ad esempio uno per Alexa) e la procedura automatica non c'è, può capitare di dover inserire un IP a mano: un tecnico deve sapere queste cose.

8 Far lampeggiare i LED

8.1 Prima prova: accendo e spengo

Per farli lampeggiare li accendo e poi li spengo: per **spegnerli tutti** scrivo tutti zeri, `PORTB = 0x00`. Dentro il `while` il programma accende, spegne e ricomincia da capo: accendo, spengo, accendo, spengo...

```
while (1)
{
    PORTB = 0xD2;
    PORTB = 0x00;
}
```

8.2 Accendere un LED alla volta

Un LED fermo acceso serve a poco: facciamolo lampeggiare. L'idea è accendere **un LED alla volta**, da RB0 a RB7: ogni volta un solo bit a 1, che si sposta. In esadecimale i valori sono `0x01`, `0x02`, `0x04`, `0x08`, `0x10`, `0x20`, `0x40`, `0x80`.

```
while (1)
{
    // accendo o spengo o leggo un dato
    PORTB = 0x01;
    PORTB = 0x02;
    PORTB = 0x04;
    PORTB = 0x08;
    PORTB = 0x10;
    PORTB = 0x20;
    PORTB = 0x40;
    PORTB = 0x80;
}
```

Compilando e caricando, i LED sembrano **tutti accesi debolmente**: in realtà si accendono e si spengono, ma **talmente in fretta** che il nostro occhio non ce la fa a vederlo (e nemmeno il simulatore riesce a mostrarlo). Lo si vede con l'**oscilloscopio** di PICSimLab (menu **Modules** → **Oscilloscope**): collegando il canale 1 a un piedino (ad esempio RB4) e scegliendo una base dei tempi stretta (ad esempio **5 ms per divisione**), si vede il segnale che **sale e scende**.

8.3 Il ritardo: `__delay_ms`

Bisogna **rallentare** il passaggio da un comando all'altro con un **ritardo**: "stai fermo un attimo fino alla prossima operazione". Il tempo si può esprimere in **millisecondi** (millesimi di secondo) o in **microsecondi** (milionesimi di secondo). In Arduino si scriveva `delay`; con XC8 il comando è:

- **`__delay_ms(500);`** per i millisecondi (mezzo secondo = **500** millisecondi);
- `__delay_us(...);` per i microsecondi: **us** perché nei caratteri di programmazione non si può usare la lettera greca μ, e si usa la u.

Attenzione

Davanti a `delay` ci sono **due trattini bassi** (*underscore*), tra `delay` e `ms` **uno solo**: `__delay_ms`. In aula il docente l'aveva scritto prima con un solo trattino ("passo tra 14 linguaggi diversi e ogni volta devo ricollegare il cervello"): il compilatore lo segnala come errore. Quando è scritto giusto, MPLAB X lo **colora** diversamente.

8.4 La frequenza del quarzo: `_XTAL_FREQ`

Che cosa ne sa il microcontrollore di quanto dura un millisecondo? Pensate al **motore di un'auto** che va a 80 km/h: quanti giri fa? Dipende dalla marcia. Allo stesso modo, il microcontrollore misura il tempo **contando operazioni**, e le operazioni sono scandite dal **clock**: con un clock veloce servono tante operazioni per fare un millisecondo, con uno lento ne bastano meno. Quindi bisogna dire alla macchina a **che frequenza va**, con un'etichetta da scrivere **in cima** al file:

```
#define _XTAL_FREQ 8000000
```

In PICSimLab il campo **Clk (MHz)** propone 20, 16, 10, 8, 4… e anche valori più alti, ma il manuale del PIC16F628A arriva **al massimo a 20 MHz**: restiamo onesti e scegliamo **8 MHz**. “Mega” vuol dire 10^6, cioè sei zeri: 8 MHz = **8 000 000** Hz.

Attenzione

Contate gli zeri. 20 MHz si scrive `20000000`: il 20 seguito da **sei zeri**, cioè **sette zeri in tutto**. Se ne scrivete uno in meno (`2000000`) state dicendo 2 MHz: il programma calcola i ritardi per una macchina **10 volte più lenta** di quella vera, e tutti i tempi risultano sbagliati. E il valore di `_XTAL_FREQ` deve essere **uguale** a quello del quarzo montato: se nel codice dite 8 MHz e nel simulatore (o sulla scheda) c’è un quarzo da 4 MHz, la macchina non si comporta **in modo corretto** (i tempi raddoppiano).

8.5 Il programma completo

```
#pragma config FOSC = HS
#pragma config WDTE = OFF
#pragma config PWRTE = ON
#pragma config MCLRE = ON
#pragma config BOREN = ON
#pragma config LVP = OFF
#pragma config CPD = OFF
#pragma config CP = OFF

#include <xc.h>

#define _XTAL_FREQ 8000000

void main(void)
{
    // questa assegnazione imposta tutti i pin B come uscite
    TRISB = 0;

    while (1)
    {
        // accendo un LED alla volta, da RB0 a RB7
        PORTB = 0x01;
        __delay_ms(500);
        PORTB = 0x02;
        __delay_ms(500);
        PORTB = 0x04;
        __delay_ms(500);
        PORTB = 0x08;
        __delay_ms(500);
        PORTB = 0x10;
        __delay_ms(500);
        PORTB = 0x20;
        __delay_ms(500);
        PORTB = 0x40;
        __delay_ms(500);
        PORTB = 0x80;
        __delay_ms(500);
    }
}
```

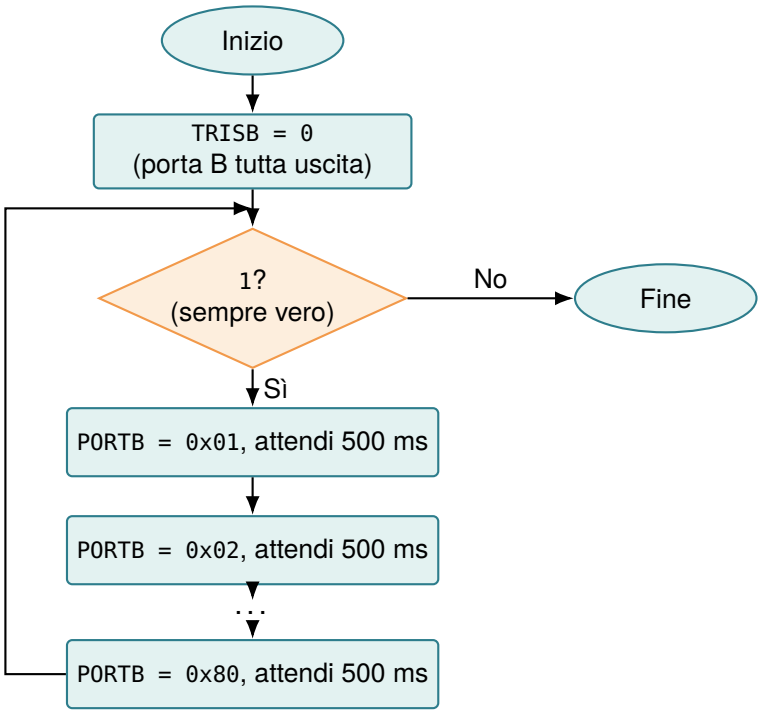


Figura 5: Diagramma di flusso del lampeggio. La condizione del `while(1)` è sempre vera: il ramo "No" e il "Fine" non vengono mai raggiunti, il programma non finisce mai.

Attenzione

Errori trovati in aula quando "non va":

- `_XTAL_FREQ` scritto in modo diverso: in minuscolo, oppure `XTAL_FRAQ`, `XTALE_FREQ`... Va scritto **esattamente così**, tutto maiuscolo, con il trattino basso davanti;
- `dalay` invece di `delay`;
- frequenza del simulatore diversa da quella scritta nel codice (8 nel codice, 4 nel simulatore);
- file **non salvato**: prima di compilare, **Ctrl+S** (o le due dischette in alto, o File → Save). Un file non salvato ha il nome **in grassetto**;
- una riga mancante (ad esempio un valore saltato nella sequenza 1, 2, 4, 8...).

Nota

Il `#define` lo spiegheremo meglio più avanti: per ora basta sapere che `_XTAL_FREQ` è un'**etichetta** che va definita e che dipende dalla velocità del quarzo.

9 Installare gli strumenti a casa

Per poter lavorare a casa (e fare il compito C01) servono tre programmi. Usiamo **versioni precise**, non le ultime: **MPLAB X v6.20** e **XC8 v3.10** sono versioni precedenti, scelte per **compatibilità** con il PIC16F628A e con gli strumenti del laboratorio. Di PICSimLab si prende invece l'**ultima versione**.

Programma	Versione	Dove scaricarlo
MPLAB X IDE	**6.20**	pagina degli archivi Microchip, tabella *MPLAB X IDE Archives*, riga v6.20 (Windows): `https://www.microchip.com/en-us/tools-resources/archives/mplab-ecosystem`
MPLAB XC8	**3.10**	pagina del compilatore XC8, sezione *Downloads* (se la 3.10 non è più tra le versioni correnti, cercarla nella pagina degli archivi qui sopra): `https://www.microchip.com/en-us/tools-resources/develop/mplab-xc-compilers/xc8`
PICSimLab	**0.9.3** (ultima, 20/09/2026)	pagina delle versioni su GitHub, file `PICSimLab_0.9.3_260920_win64_setup.exe`: `https://github.com/lcgamboa/picsimlab/releases`

Scheda del microcontrollore (manuale e caratteristiche): `https://www.microchip.com/en-us/product/PIC16F628A`.

1. Installate **prima MPLAB X**, poi **XC8**, poi **PICSimLab**. In aula il docente ha estratto dalla vecchia dispensa “MPLAB X” (pagine 51–69 della dispensa “Micro PIC 8 bit e Timer”) le pagine con i link per scaricare i programmi: indicavano MPLAB X 6.15, XC8 2.45 e PICSimLab 0.9.0. **Non usate quelle versioni**, usate quelle della tabella.
2. Lo scaricamento è meno di un giga, ma una volta installati i programmi occupano circa **8–10 GB** sul disco.
3. Durante l’installazione di **XC8**, nella schermata *Compiler Settings*, lasciate **spuntate tutte le opzioni**, in particolare **Add xc8 to the PATH**.

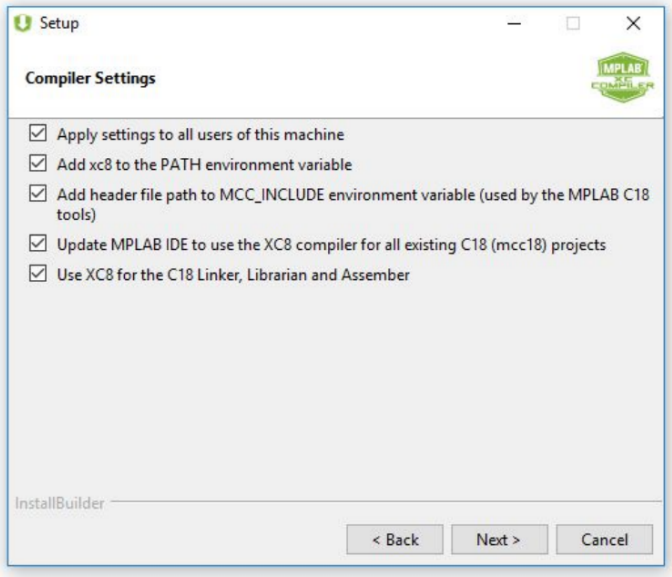


Figura 6: Installazione di XC8, schermata *Compiler Settings*: tutte le opzioni spuntate (immagine dalla dispensa “Micro PIC 8 bit e Timer” degli anni precedenti).

Nota

Il docente ci tiene: **entro marzo** dovete saper programmare il PIC, perché l'anno prossimo lo userete per il **progetto dell'esame**, e dovete essere in grado di portarlo avanti da soli. Questo è il primo compito per casa, e non sarà l'ultimo.

10 Cosa ricordare di questa lezione

- Il PIC ha un'architettura **Harvard**: **program memory** e **data memory** separate (2048 parole e 224 byte nel PIC16F628A). Il PC usa la **Von Neumann**: una sola memoria per programma e dati.
- I piedini sono raggruppati in porte da 8 (A e B). **TRIS** decide la direzione (**1** = Input, **0** = Output), **PORT** contiene il valore: a sinistra dell'uguale si **scrive**, altrove si **legge**.
- Nuovo progetto in MPLAB X: **PIC16F628A** (con la A), nessun debugger, compilatore **XC8**, nomi senza spazi né punti, file **`main.c`** in Source Files.
- Configuration bits: **FOSC = HS**, **WDTE = OFF**, PWRTE, MCLRE, BOREN **ON**, LVP, CPD, CP **OFF**. Le 8 righe `#pragma` vanno in cima al file.
- Compilare in MPLAB X 6.20: **Build Main Project** (dalla freccina o dal pulsante aggiunto alla toolbar). Simulare: PICSimLab, scheda **McLab1**, File → **Load Hex**, deviatore su **LED**.
- In un dispositivo embedded il programma **non deve mai finire**: **`while (1)`**. Solo lo zero è falso.
- Lo stesso numero si scrive in decimale (`82`), binario (`0b01010010`) o esadecimale (`0x52`): dentro la macchina è sempre binario. Ogni **4 bit** corrispondono a **una cifra esadecimale**.
- Per i ritardi: **`__delay_ms(...)`** (due trattini davanti) e **`#define _XTAL_FREQ 8000000`** uguale alla frequenza del quarzo.

Esempio

Compito per casa e verifica. Il gioco di luci deve essere **fatto da voi**, non da ChatGPT: se un'intelligenza artificiale vi scrive una cosa sbagliata, **sapete correggerla**? Lo scopo è imparare a farlo. Il compito per casa (dispensa **C01-A04**: installare gli strumenti e scrivere un gioco di luci) va consegnato **entro giovedì 29 ottobre** ed è **una parte della Verifica 1**, insieme alla verifica in aula dello stesso giorno.